



■ RAI PUBBLICITÀ · UPSKILLING · SESSION 3 OF 3

Agentic AI on Microsoft Foundry

Day 1 — Vision, Architecture & Live Demos

Agent Framework · Foundry Hosted Agents · MCP · A2A

Mauro Minella

Senior Cloud Solution Architect · Microsoft

Welcome — where we are in the journey

SESSION 1

Foundations (AI Platform)

SESSION 2

Building blocks (evaluation)

SESSION 3 — TODAY

Agentic AI



Who's in the room

- Group A — Data Center, Tech Evolution, Architecture, Agentic AI Modernization & AI Governance leads
- Group B — Cloud Solution Architects, Developers, Data Scientists
- Today we speak to both — vision first, then real engineering



Two days, one arc

- Day 1 (today): the map, the architectures, live demos for everyone
- Day 2: technical deep dive, deeper demos and hands-on exercises for Group B
- By the end you can place every piece on the board

Software that acts — not just answers



YESTERDAY

Applications we operate

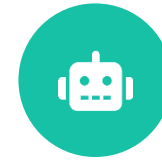
We click, type and drive every step ourselves.



TODAY

Assistants that answer

They understand language and reply — but wait for us.



NOW

Agents that get work done

They reason about a goal and take action, step by step.

The big shift: from tools we operate to digital colleagues that act on our behalf.

Why now — and why it matters for RAI



Models can reason & act

- Frontier models plan and call tools reliably
- Grounding on your data, not just training



Everything is connectable

- Open protocols (MCP, A2A) link tools & agents
- Compose across vendors, no lock-in



Enterprise-ready

- Identity, governance, safety built in
- Runs where your data & rules live

The opportunity for RAI Pubblicità

- Automate real workflows across advertising sales, campaign operations and media planning
- Turn scattered knowledge — SharePoint, data, systems — into self-service, governed assistants
- Free experts from repetitive back-office work, with a full audit trail behind every action

Agenda — our 4 hours together

01 The Agentic Landscape & Microsoft's Vision

 *Coffee break · reposition & reset*

02 Microsoft Agent Framework

03 Microsoft Foundry & Hosted Agents

 *Coffee break · reposition & reset*

04 Model Context Protocol (MCP)

05 Agent-to-Agent (A2A) Protocol

06 Putting It All Together → bridge to Day 2

Two days, two altitudes



Day 1 — today · Group A + B

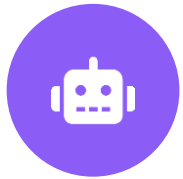
- Breadth: the full map of the agentic stack
- Vision, architectures and business opportunities
- High-level overview of each technology
- Short, ready-to-deliver live demos
- Goal: everyone can place every piece



Day 2 — Group B (technical)

- Depth: a deep dive on selected topics
- More detailed, hands-on demos
- Code, patterns and design trade-offs
- Guided exercises / labs for participants
- Goal: build and wire agents yourselves

What you'll see live today



Demo 1 · Agent Framework

"Hello Agent" that reasons and calls a function tool



Demo 2 · Foundry portal

A Prompt Agent with Code Interpreter + File Search



Demo 3 · MCP

An agent discovering and calling an MCP tool



Demo 4 · A2A

A Foundry agent delegating to a remote A2A agent

Ready-to-deliver: every demo runs on the Foundry / Azure platform — no environment setup, no data loading.

How to get the most out of today



Vision + real engineering

- Not a marketing deck: expect true architecture and code
- We alternate the 'why' for leaders and the 'how' for builders



Ask anytime

- Interrupt with questions — they shape tomorrow's deep dive
- No question is too basic or too advanced



Four names to remember

- Agent Framework · Foundry · MCP · A2A
- By tonight these will feel familiar



Hands-on comes next

- Day 1 builds the mental model
- Day 2 puts the keyboard in your hands



■ THEME 1 OF 6 · OVERVIEW + DEMO

The Agentic Landscape & Microsoft's Vision

*From chatbots to systems of collaborating agents —
and the map that ties it together.*

From chatbots to agents to systems of agents



Chatbot

Answers

- Responds to a prompt
- Stateless, no tools
- You do the work



Agent

Reasons + acts

- Plans toward a goal
- Calls tools & data
- Multi-step, autonomous



Multi-agent system

Collaborates

- Specialized agents
- Delegate & coordinate
- Solve complex tasks

More autonomy, more capability — and more need for orchestration and governance.

Anatomy of an AI agent



An agent

*reasons about a goal
and takes autonomous,
multi-step action to reach it*



Model

Reasoning & language, from the Foundry catalog



Instructions

Goals, constraints and behavior (the *prompt*)



Tools

Access to data and actions: search, APIs, code



Memory & state

Threads, context and long-term memory

The agentic loop



1. Perceive

Read the goal & context



2. Reason / plan

Decide the next step



3. Act

Call a tool or API



4. Observe

Read the result

Repeat until the goal is met, but the loop is bounded by:

- **grounding** (facts from your data),
- **guardrails** (safety & policy) and stopping conditions (max steps, cost, human approval).

From one agent to many



Why not one big agent?

- A single mega-prompt becomes brittle and hard to test
- Too many tools confuse model tool-selection
- One failure sinks the whole task



Why specialized agents

- Separation of concerns — each agent does one thing well
- Parallelism, reuse and independent evolution
- Resilience: isolate and retry a single step

Common coordination shapes (we'll detail them in Theme 2)



Orchestrator + workers



Handoff between agents

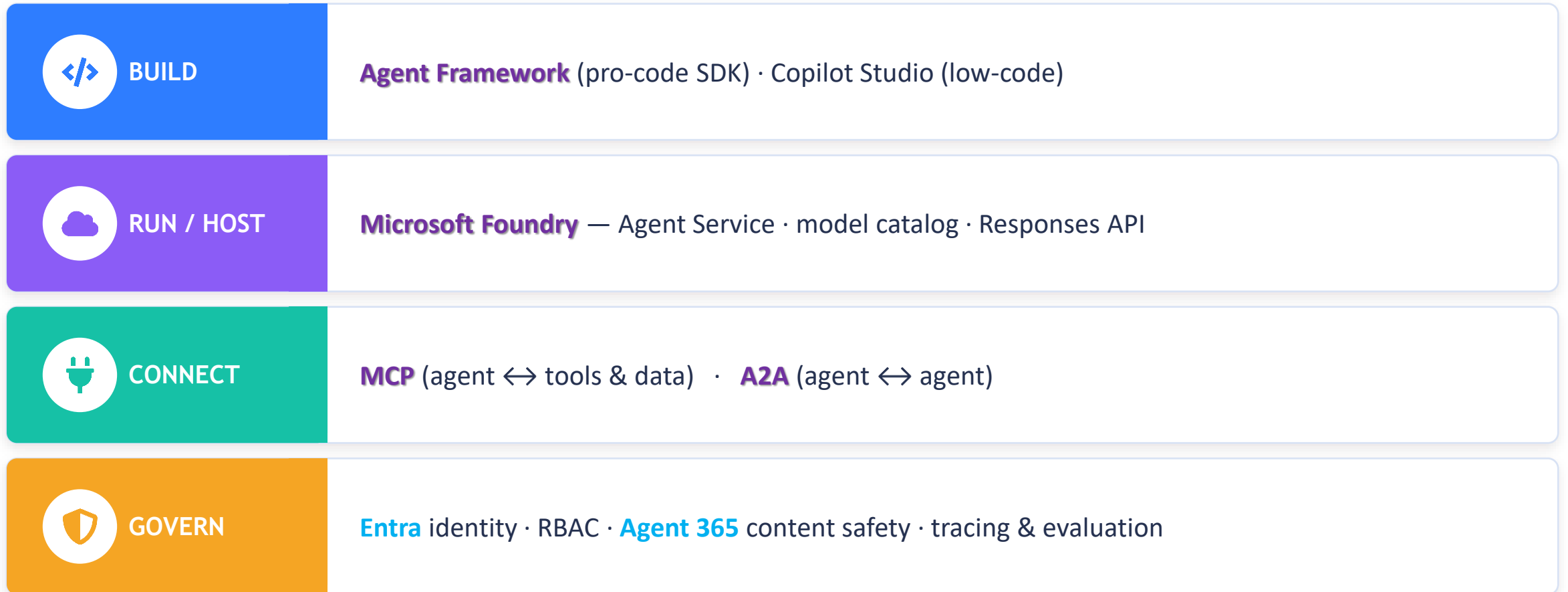


Group chat / debate



Sequential pipeline

The Microsoft agentic stack



Open by design: why protocols matter

Without standards

- Every model $\times$ every tool = custom glue
- $M \times N$ integrations to build and maintain
- Fragile, duplicated, vendor-locked
- New tool $\rightarrow$ rewrite for every agent

With open protocols

- MCP + A2A turn $M \times N$ into $M + N$
- Build a tool / agent once — reuse everywhere
- Compose across vendors — no lock-in
- Microsoft embraces the open ecosystem

Where this creates value



Faster operations

Automate campaign ops, order handling and media planning steps



Self-service knowledge

Governed assistants over SharePoint, data and internal systems



Back-office automation

Offload repetitive tasks with a human in the loop where it matters



Governed & auditable

Every action traced, permissioned and policy-checked by design



■ THEME 2 OF 6 · OVERVIEW + DEMO

Microsoft Agent Framework

*One open-source SDK to build single agents and multi-agent workflows —
in .NET, Python and Go.*

The successor to Semantic Kernel + AutoGen



AutoGen

Simple abstractions for single- & multi-agent patterns



Semantic Kernel

Enterprise features: state, type-safety, middleware, telemetry



Microsoft Agent Framework

- Open-source SDK, direct successor to both — built by the same teams
- .NET and Python (GA) · Go in public preview
- AutoGen's simplicity + Semantic Kernel's enterprise backbone
- Works with Foundry, Azure OpenAI, OpenAI, Anthropic, Ollama...

Three capabilities in one framework



Agents

- An LLM that processes input, calls tools & MCP servers, and responds
- The autonomous, conversational unit



Workflows

- Graph-based orchestration of agents & functions
- Type-safe routing, checkpointing, human-in-the-loop



Harness

- An opinionated agent for long, multi-step tasks
- Planning, memory, tool-approval, observability built in

The building blocks



Model clients

Chat-completions & responses APIs across providers



Context providers

Pluggable memory the agent can read & write



MCP clients

Attach external tools via the Model Context Protocol



Agent session

State management for multi-turn conversations



Middleware

Intercept and govern agent actions (filters, policy)



Tools / functions

Your code exposed to the agent as callable actions

Agents vs Workflows — when to use which



Use an AGENT when...

- The task is open-ended or conversational
- You need autonomous tool use and planning
- A single LLM call (with tools) can suffice
- The path isn't known in advance



Use a WORKFLOW when...

- The process has well-defined steps
- You need explicit control over order
- Multiple agents / functions must coordinate
- You want checkpoints & human approval

Multi-agent orchestration patterns



Sequential

Agents run in a fixed pipeline; each output feeds the next



Concurrent

Agents work in parallel; results are aggregated



Group chat

Agents converse and debate toward a shared answer



Handoff

An agent delegates control to a better-suited agent



Magentic

A manager agent plans, assigns and tracks the work



Custom (workflow)

Graph you design: branches, loops, human steps



A first agent, in code

C# (.NET)

```
using Azure.AI.Projects;
using Azure.Identity;
using Microsoft.Agents.AI;

AIAgent agent = new AIProjectClient(
    new Uri(foundryProjectEndpoint),
    new AzureCliCredential())
    .AsAIAgent(
        model: "gpt-5-mini",
        instructions: "You are a
            concise assistant.");

Console.WriteLine(
    await agent.RunAsync(
        "Largest city in France?"));
```

Python

```
from agent_framework.foundry import FoundryChatClient
from azure.identity import AzureCliCredential

client = FoundryChatClient(
    project_endpoint=ENDPOINT,
    model="gpt-5-mini",
    credential=AzureCliCredential())

agent = client.as_agent(
    name="HelloAgent",
    instructions="You are a concise assistant.")

await agent.run("...")
```

The same agent runs locally today and as a Foundry-hosted agent tomorrow — no rewrite.



Same answer either way — but MAF gives you an agent

OpenAI — direct call

```
client = AzureOpenAI(  
    azure_ad_token_provider=token,  
    azure_endpoint=ENDPOINT)  
  
resp = client.responses.create(  
    model=DEPLOYMENT, input=query)  
  
print(resp.output_text)
```

→ replies to the query directly

Microsoft Agent Framework

```
client = OpenAIChatClient(  
    model=DEPLOYMENT,  
    credential=AzureCliCredential())  
  
agent = Agent(client=client,  
    name="Assistant",  
    instructions="You are a clever agent.")  
  
resp = await agent.run(query)
```

→ same answer — now from an agent

Identical output for a plain query — but MAF already gives you a named, instructed agent, ready to add tools and memory.



Add a tool: OpenAI returns a call, MAF runs it

OpenAI – direct call

```
resp = client.responses.create(
    model=DEPLOYMENT, input=query,
    tools=[protocol_tool_schema])

# the model does NOT run the tool
resp.output_text      → empty
resp.output[0].name   → tool to call
resp.output[0].call_id → call id
```

→ only a tool-call request – you run it, then loop back

Microsoft Agent Framework

```
def ProtocolNumberGenerator():
    """Generate a protocol number."""
    return str(random.randint(...))

agent = Agent(client=client,
    instructions="Start with the protocol #.",
    tools=[ProtocolNumberGenerator])

resp = await agent.run(query)
```

→ harness runs the tool, returns the final answer

OpenAI returns the tool call for you to run; the MAF harness executes it and returns the finished answer.



■ THEME 3 OF 6 · OVERVIEW + DEMO

Microsoft Foundry Hosted Agents

*The managed platform that runs, scales, secures and observes
your agents in production.*

Two agent types: Prompt vs Hosted



Prompt agents

- Defined by configuration: instructions + model + tools
- Authored in the portal, or via SDK / REST
- Fully managed — no code, no compute to run
- Best for: fast start, production agents without custom orchestration



Hosted agents

- Your code: Agent Framework, LangGraph, OpenAI/Anthropic SDK...
- Packaged as a container or a zip of source
- Foundry runs it: managed endpoint, autoscale, Entra identity, state
- Best for: custom code, custom orchestration, multi-agent

Hosted Agents architecture



An app, not a config

A prompt agent is declarative JSON; a hosted agent is real application code



Any framework

Agent Framework, Semantic Kernel, AutoGen, LangGraph — or plain code



An LLM is optional

Arbitrary logic: your agent need not call a model at all



Agent Server

The wrapper that makes your code callable as a Foundry agent



Container-based

A Docker image in your registry — the skills your team already has



One agent outside

Any internal complexity, a single managed agent to every caller

Building Hosted Agents with Agent Server



What Agent Server gives you

1. Templates that receive the calls Foundry sends your agent
2. A hosting adapter that speaks the Foundry agent contract
3. Run and debug the agent locally, before publishing
4. Tooling to push and run the container on Foundry



How you build one

1. Write the agent logic in your framework of choice
2. Wrap it with the Agent Server entry point
3. Add a Dockerfile — or let Foundry build from a source zip
4. Push the image to a registry such as ACR

Agent Server is not Agent Framework: one writes the agent, the other makes it consumable by Foundry.

Publishing Hosted Agents to Foundry



Push the image

Container image in ACR, or a zip of your source



Create the agent

Declare the hosted agent in your Foundry project



Foundry runs it

Managed endpoint, autoscale, dedicated identity

After publishing



Called like a prompt agent



Same Responses API contract



Versioned by image tag



Traced and evaluated in the project

From the caller's side, a hosted agent and a prompt agent are indistinguishable.

Agent Service — bring your framework, we run it



Your code

Agent Framework /
LangGraph / custom



Package

Container image or a zip of
source



Foundry runs it

Managed endpoint +
autoscale

What Foundry adds for free



Managed endpoint



Autoscale per session



Dedicated Entra identity



Session state



End-to-end tracing



Responses API + tools



Building a hosted agent with Microsoft Agent Toolkit

1 · Scaffold

Install the Foundry Toolkit extension in VS Code

Developer Tools > Build >
Create Agent

Browse all samples →
Python · Agent Framework ·
Responses → Basic Hosted Agent

→ project files: `azure.yaml`, `main.py`,
`requirements.txt`

2 · Run locally

```
pip install -r requirements.txt
```

```
check .env:  
FOUNDRY_PROJECT_ENDPOINT  
AZURE_AI_MODEL_DEPLOYMENT_NAME
```

```
az login
```

```
F5 → Debug Local Agent Server
```

→ Agent Inspector on :8088 – chat,
breakpoints, traces

3 · Deploy & call

```
Build > Deploy to Foundry
```

```
Code + Remote package mode  
New agent · Python 3.13  
entry point: python3 main.py
```

```
POST /responses  
{"input": "Hello"}
```

→ managed endpoint, versions, sessions,
traces

Same code, three moments: scaffold it, prove it on your laptop, then let Foundry run it.



THEME 4 OF 6 · OVERVIEW + DEMO

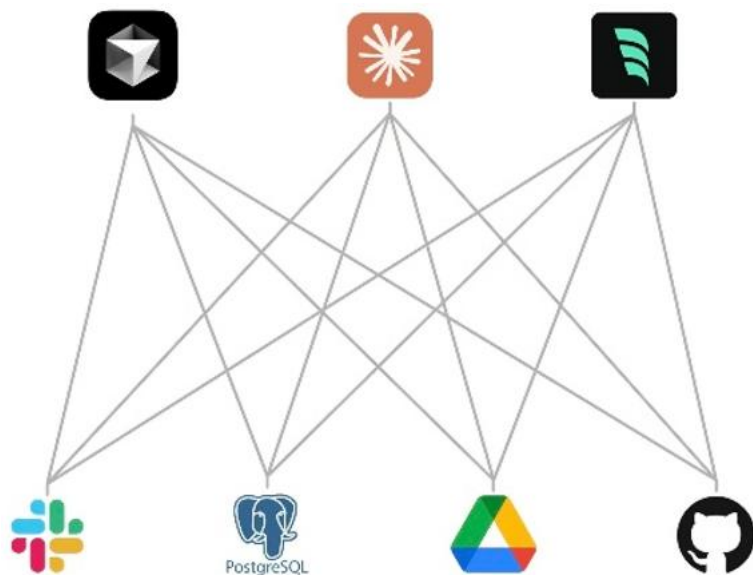
Model Context Protocol (MCP)

The open standard that lets any agent connect to tools and data — the USB-C of AI.

the problem it solves, what it is, how it works, core building blocks, how Microsoft has adopted it, security / governance considerations, and third demo of the day

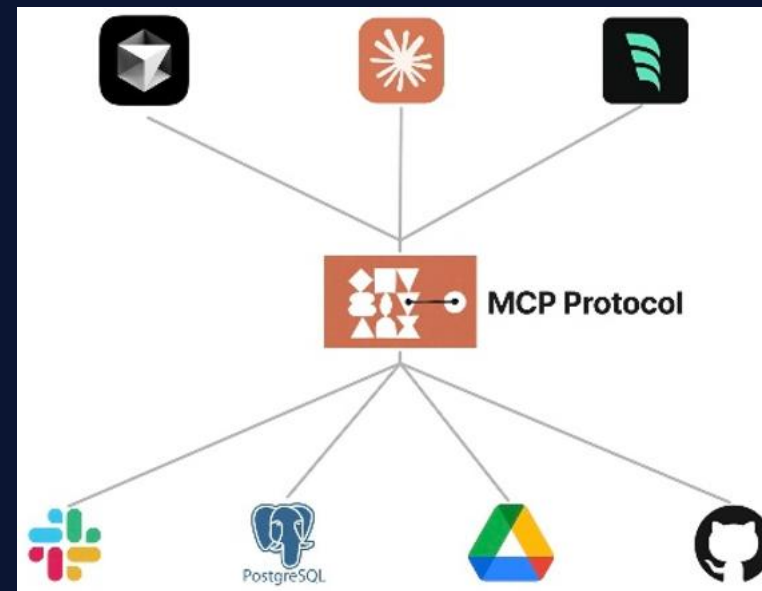
The $M \times N$ integration problem

Before — custom glue



$M \times N$ connectors

After — one protocol



$M + N$ connectors

What is the Model Context Protocol?



An open standard

- Introduced by Anthropic, now broadly adopted
- Vendor-neutral — an ecosystem, not a product
- A fast-evolving spec the whole industry builds on



Client-server over JSON-RPC

- The agent hosts an MCP client
- A server exposes capabilities to it
- Transport: stdio (local) or HTTP/SSE (remote)



Host / Client
(the agent)

JSON-RPC · tools / resources / prompts



MCP Server
(tools & data)

MCP primitives



Tools

Callable actions the agent can invoke

`"createInvoice(...)", "queryDb(...)"`



Resources

Readable data & context to ground on

`files, records, API responses`



Prompts

Reusable, parameterized templates

`"summarize-campaign", "triage"`



Sampling

Server can ask the client's model to reason

`server-initiated completions`

MCP across the Microsoft ecosystem



Foundry Agent Service

Add remote MCP servers from the tool catalog (e.g. Azure DevOps MCP)



Custom MCP on Azure Functions

Expose your own tools via the Functions MCP webhook endpoint



Toolbox

Curated tools behind one versioned, MCP-compatible endpoint



Agent Framework

Built-in MCP clients — attach servers directly in code



Copilot Studio

Low-code agents consume MCP tools too



Windows & GitHub

MCP support reaching the developer & OS surface

MCP: security & governance



Authentication options

- Key-based access
- Microsoft Entra (agent / project managed identity)
- OAuth On-Behalf-Of (identity passthrough)



Control what agents can do

- Expose only a curated subset of tools
- Least-privilege scopes per connection
- Consent & approval before sensitive actions



Watch the trust boundary. A tool can return text that tries to hijack the agent — prompt injection, including cross-prompt injection (XPIA). Treat external MCP servers as untrusted: validate inputs/outputs, apply content safety, keep a human in the loop for high-impact actions, and know the spec keeps evolving on auth & security.

Demo – Expose the campaigns as an MCP server

- | | |
|--|---|
| <ul style="list-style-type: none">• Explore the architecture of an MCP server<ul style="list-style-type: none">• Tools for actions and computations• Resources for data and context• Prompts for reusable interaction templates• Connect to the server with a Python MCP client<ul style="list-style-type: none">• Discover exposed components• Invoke tools and retrieve resources and prompts• Separate MCP from AI<ul style="list-style-type: none">• MCP is a specification and protocol• No agent or LLM is required to use it | <ul style="list-style-type: none">• Add natural-language interaction through an LLM<ul style="list-style-type: none">• GitHub Copilot in VS Code• Microsoft Agent Framework• Microsoft Foundry SDK• Build once, reuse everywhere<ul style="list-style-type: none">• The same MCP server supports multiple clients and agentic experiences |
|--|---|



■ THEME 5 OF 6 · OVERVIEW + DEMO

Agent-to-Agent (A2A) Protocol

How agents discover, delegate to and collaborate with other agents — across teams and vendors.

Why we need it, main features, differences vs MCP, how Microsoft supports it, demo 4.

Beyond tools: agents that call agents



Some capabilities are agents

- Owned by another team or vendor
- Have their own model, tools and data
- You want the outcome, not their code



Delegate, don't absorb

- Call a remote agent like a colleague
- Keep systems decoupled & independently owned
- Compose specialists into one experience

Example for AdverSphere Broadcasting

A sales assistant agent asks a specialized pricing agent for a quote, and a media-planning agent for availability — each owned by a different team, composed into one answer.

What is the A2A protocol?



An open protocol

- Originated by Google, now under the Linux Foundation
- Vendor-neutral standard for agent interoper
- Complements MCP — different job (see next)



Client agent ↔ remote agent

- A client agent delegates a task
- A remote agent performs and responds
- Tasks & messages, with streaming updates

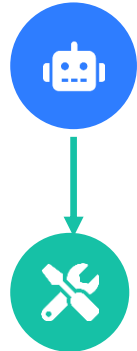


The Agent Card. Every A2A agent publishes a machine-readable card describing who it is, what skills it offers, its endpoint and how to authenticate — so other agents can discover it and know how to delegate.

Discovery + capabilities + identity, in one document.

MCP vs A2A — complementary, not competing

MCP agent ↔ tools & data



Vertical — reach down to a capability

A2A agent ↔ agent



Horizontal — delegate across peers

Together: multi-agent solutions can use MCP tools to act and A2A to delegate to other agents. You almost always use both.

A2A in Microsoft Foundry & Agent Framework



Foundry A2A tool (preview)

Connect a Foundry agent to a remote A2A endpoint and delegate tasks



Agent Framework A2A

Built-in A2A integration to expose or consume agents in code

Sharing context across agents.

The A2A tool makes it easy to share context between Foundry-model-powered agents and external agent endpoints through a standardized protocol — publishing agents to Microsoft 365 Copilot, Teams and the Entra Agent Registry.

Publishing an A2A agent – MAF removes the boilerplate

Plain A2A SDK – with AgentExecutor

```
# --- Agent logic ---
def quote(brief: str) -> str:
    return f"You asked a quote for: {brief}"

# --- Minimal Agent Card ---
agent_card = AgentCard(
    name="Minimal A2A Agent with Agent Executor.",
    description="Ultra-minimal A2A demo.",
    version="1.0",
    supported_interfaces=[AgentInterface(url="http://localhost:9999/",
                                         protocol_binding="JSONRPC")]
)

# --- Agent executor ---
class PricingAgent(AgentExecutor):
    async def execute(self, context: RequestContext, queue: EventQueue):
        await queue.enqueue_event(
            new_text_message(quote(context.get_user_input()))
        )
    async def cancel(self, *_):
        pass

# --- Server ---
handler = DefaultRequestHandler(
    agent_executor=PricingAgent(),
    task_store=InMemoryTaskStore(),
    agent_card=agent_card,
)

app = Starlette(routes=[*create_agent_card_routes(agent_card),
                       *create_jsonrpc_routes(handler, rpc_url="/")])

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=9999)
```

Microsoft Agent Framework – no executor

```
# --- Agent logic ---
def quote(brief: str) -> str:
    return f"Preventivo per: {brief}"

# --- Minimal Agent Card ---
agent_card = AgentCard(
    name="Minimal A2A Agent with Microsoft Agent Framework.",
    description="Ultra-minimal A2A demo.",
    version="1.0",
    supported_interfaces=[AgentInterface(url="http://localhost:9999/",
                                         protocol_binding="JSONRPC")]
)

# --- Agent creation with Microsoft Agent Framework ---
agent = Agent(
    client=OpenAIChatClient(
        model=os.environ["AZURE_OPENAI_CHAT_DEPLOYMENT_NAME"],
        credential=AzureCliCredential(),
    ),
    name="PricingAgent",
    instructions="Use the tool to calculate the quote (il tool quote).",
    tools=[quote],
)

# --- Server ---
handler = DefaultRequestHandler(
    agent_executor=A2AExecutor(agent),
    task_store=InMemoryTaskStore(),
    agent_card=agent_card,
)

app = Starlette(routes=[*create_agent_card_routes(agent_card),
                       *create_jsonrpc_routes(handler, rpc_url="/")])

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=9999)
```

Consuming A2A agents — a loop of calls, or one workflow

Plain A2A client

```
import asyncio
from a2a.client import create_client
from a2a.helpers import new_text_message
from a2a.types import Role, SendMessageRequest

CAMPAIGN_BRIEF = "sector=Travel; impressions=9200000"

async def main():
    client = await create_client("http://localhost:9999")
    async with client:
        request = SendMessageRequest(
            message=new_text_message(
                CAMPAIGN_BRIEF,
                role=Role.ROLE_USER,
            )
        )

        async for response in client.send_message(request):
            print(response.message.parts)

asyncio.run(main())
```

Microsoft Agent Framework client

```
import asyncio
from agent_framework.a2a import A2AAgent

CAMPAIGN_BRIEF = "sector=Travel; impressions=9200000"

async def main():
    pricing_agent = A2AAgent(
        name="PricingAgent",
        description="Computes an advertising campaign quote.",
        url="http://localhost:9999",
    )

    async with pricing_agent:
        result = await pricing_agent.run(CAMPAIGN_BRIEF)

    print(f"Pricing: {result.text}")

asyncio.run(main())
```

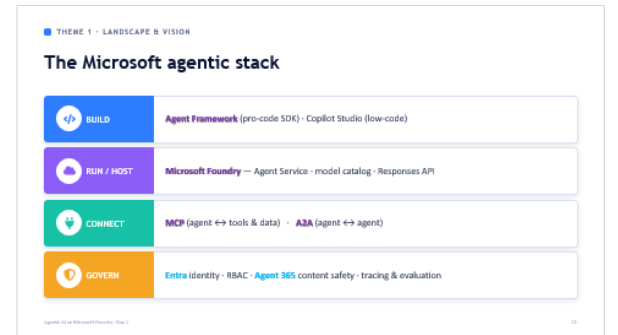


■ THEME 6 OF 6 · SYNTHESIS + BRIDGE

Putting It All Together

One reference architecture that unites the four technologies — and a bridge to Day 2.

A reference architecture



Experience

M365 Copilot · Teams · your apps **meet the Agent**



Built with

Agent Framework — agents & workflows



Hosted & run by

Microsoft Foundry — Agent Service, model catalog, Responses API



Connected via

MCP → tools & data (SharePoint, AI Search, custom) · *A2A* → other agents



Governed by

Entra identity · RBAC · content safety · tracing & evaluation

How the four pieces relate



Agent Framework

HOW you build

The open SDK for agents & multi-agent workflows



Microsoft Foundry

WHERE it runs

The managed platform that hosts, scales & governs



MCP

How it CONNECTS to tools

One standard to reach tools & data



A2A

How it TALKS to agents

One standard to delegate across agents

Five things to remember



1

An agent = model + instructions + tools, running in a loop



2

Agent Framework is HOW you build — open, multi-language



3

Foundry is WHERE it runs — managed, scaled, governed



4

MCP connects agents to tools & data — one open standard



5

A2A lets agents collaborate — open, and identity-aware

Resources & next steps



Microsoft Agent Framework

learn.microsoft.com/agent-framework



Foundry Agent Service

learn.microsoft.com/azure/ai-foundry/agents



Model Context Protocol

modelcontextprotocol.io



A2A in Foundry

learn.microsoft.com/azure/foundry/agents/how-to/tools/agent-to-agent



Samples & quickstarts

github.com/microsoft/agent-framework



Community

Microsoft AI Discord

Tomorrow — deep dive & hands-on (Group B)



Agent Framework in depth

Workflows & orchestration in code; tools, middleware, state



Foundry hosted agents

Package, deploy and scale a real hosted agent end-to-end



Build & secure an MCP server

Expose your own tools; auth, scopes and safety



Wire up A2A

Publish an Agent Card and delegate across agents



Longer, deeper demos + guided exercises — you build and wire agents yourselves.



Questions?

Grazie — see you tomorrow for the technical deep dive & hands-on labs.

Mauro Minella

Senior Cloud Solution Architect · Microsoft

Mauro.Minella@microsoft.com